

CSCI-4208: Developing Advanced Web Applications

Week 6: Lecture 9

JavaScript - DOM

DOM - Overview

1. Browser environment, specs
2. DOM tree
3. Walking the DOM
4. Searching: getElement*, querySelector*
5. Node properties: type, tag and contents
6. Attributes and properties
7. Modifying the document
8. Styles and classes
9. Element size and scrolling
10. Window sizes and scrolling
11. Coordinates

1. DOM: Browser environment, specs

JavaScript was designed as a *Domain Specific Language* created for web browsers. It's evolved into a *General Purpose Language* with many uses and platforms. The JS platforms we'll learn about are browsers & web-servers.

Each JS platform (*or host environment*) has its own specific functionality (*i.e. its own objects and functions additional to the language core.*)

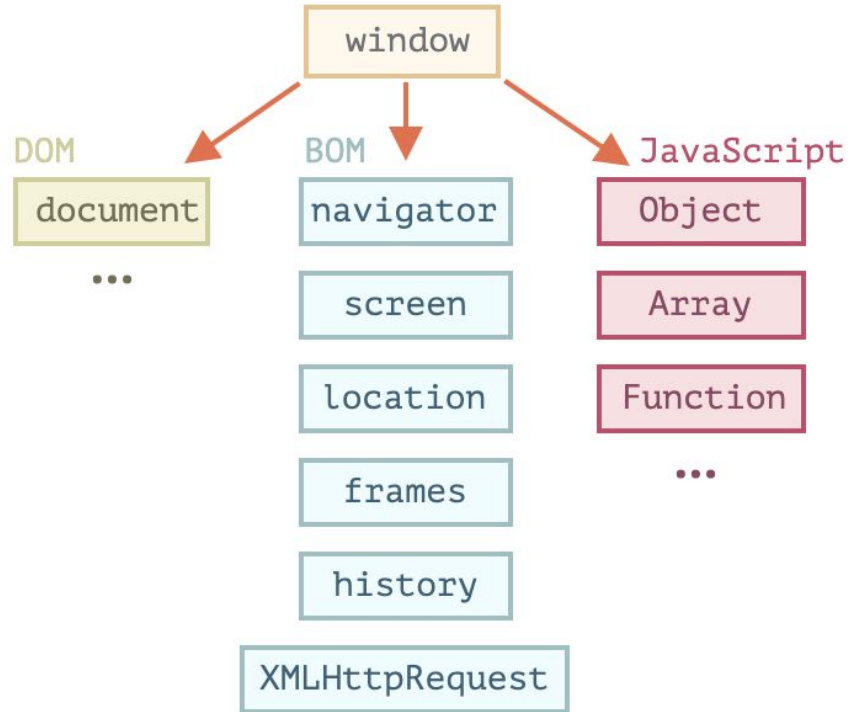
- Web browsers give a means to control web pages.
- Node.js provides server-side features.

1. DOM: Browser environment, specs

Here's a bird's-eye view of what we have when JavaScript runs in a web browser:

There's a “root” object called window. It has two roles:

1. Its a global object for JavaScript code.
2. It represents the “browser window” and provides methods to control it.



1. DOM: Browser environment, specs

For instance, here we use it as a global object:

Role 1:

```
1 function sayHi() {  
2   alert("Hello");  
3 }  
4  
5 // global functions are methods of the global object:  
6 window.sayHi();
```

And here we use it as a browser window, to see the window height:

Role 2:

```
1 alert(window.innerHeight); // inner window height
```

There are more window-specific methods and properties, we'll cover them later.

1. DOM: Browser environment, specs

DOM (Document Object Model)

Document Object Model, or DOM for short, represents all page content as objects that can be modified.

The `document` object is the main “entry point” to the page. We can change or create anything on the page using it.

For instance:

```
1 // change the background color to red
2 document.body.style.background = "red";
3
4 // change it back after 1 second
5 setTimeout(() => document.body.style.background = "", 1000);
```

Here we used `document.body.style`, but there's much, much more. Properties and methods are described in the specification: [DOM Living Standard](#).

1. DOM: Browser environment, specs

BOM (Browser Object Model)

The Browser Object Model (BOM) represents additional objects provided by the browser (host environment) for working with everything except the document.

Specifications

BOM is the part of the general [HTML specification](#).

Yes, you heard that right. The HTML spec at <https://html.spec.whatwg.org> is not only about the “HTML language” (tags, attributes), but also covers a bunch of objects, methods and browser-specific DOM extensions. That’s “HTML in broad terms”. Also, some parts have additional specs listed at <https://spec.whatwg.org>.

1. DOM: Browser environment, specs

Summary

Talking about standards, we have:

DOM specification

Describes the document structure, manipulations and events, see <https://dom.spec.whatwg.org>.

CSSOM specification

Describes stylesheets and style rules, manipulations with them and their binding to documents, see <https://www.w3.org/TR/cssom-1/>.

HTML specification

Describes the HTML language (e.g. tags) and also the BOM (browser object model) – various browser functions: `setTimeout`, `alert`, `location` and so on, see <https://html.spec.whatwg.org>. It takes the DOM specification and extends it with many additional properties and methods.

2. DOM: DOM tree

The backbone of an HTML document is tags.

According to the Document Object Model (DOM), every HTML tag is an object. Nested tags are “children” of the enclosing one. The text inside a tag is an object as well.

All these objects are accessible using JavaScript, and we can use them to modify the page.

For example, `document.body` is the object representing the `<body>` tag.

Running this code will make the `<body>` red for 3 seconds:

```
1 document.body.style.background = 'red'; // make the background red
2
3 setTimeout(() => document.body.style.background = '', 3000); // return back
```

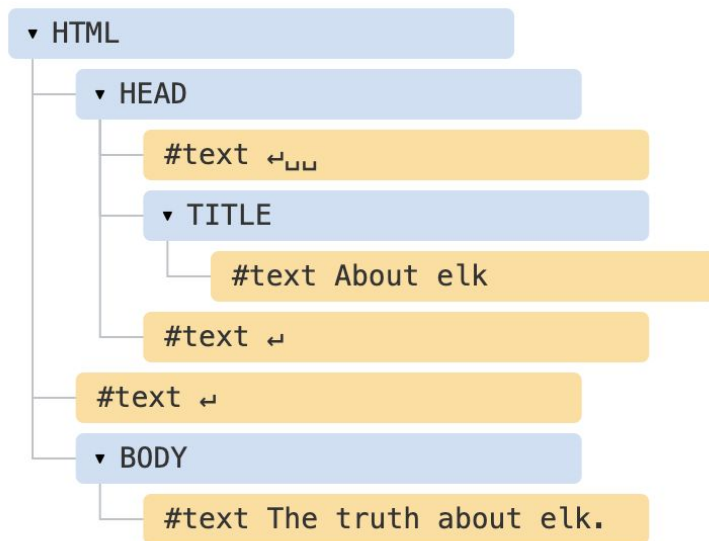
2. DOM: DOM tree

An example of the DOM

Let's start with the following simple document:

```
1 <!DOCTYPE HTML>
2 <html>
3 <head>
4   <title>About elk</title>
5 </head>
6 <body>
7   The truth about elk.
8 </body>
9 </html>
```

Element Nodes, Text Nodes



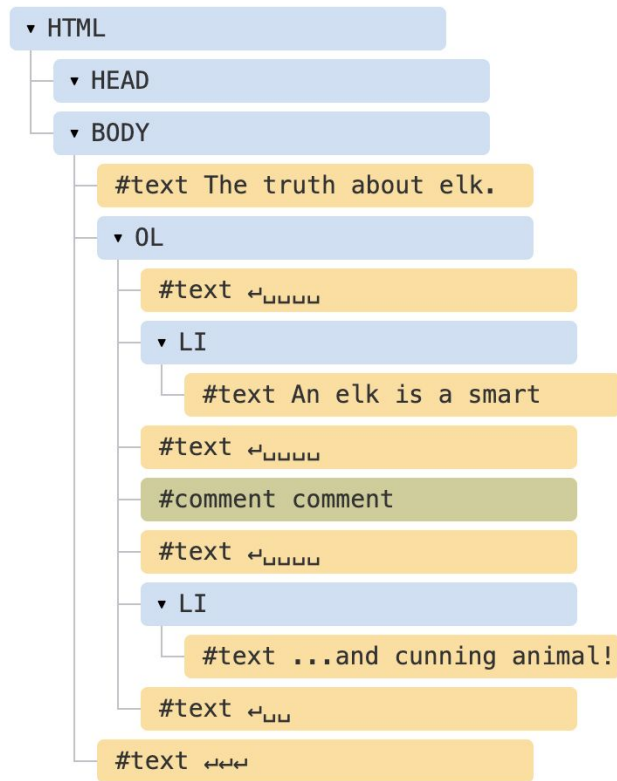
2. DOM: DOM tree

Other node types

There are some other node types besides elements and text nodes.

For example, comments:

```
1 <!DOCTYPE HTML>
2 <html>
3 <body>
4   The truth about elk.
5   <ol>
6     <li>An elk is a smart</li>
7     <!-- comment -->
8     <li>...and cunning animal!</li>
9   </ol>
10 </body>
11 </html>
```



2. DOM: DOM tree

Everything in HTML, even comments, becomes a part of the DOM.

Even the `<!DOCTYPE...>` directive at the very beginning of HTML is also a DOM node. It's in the DOM tree right before `<html>`. Few people know about that. We are not going to touch that node, we even don't draw it on diagrams, but it's there.

The `document` object that represents the whole document is, formally, a DOM node as well.

There are **12 node types**. In practice we usually work with 4 of them:

1. `document` – the “entry point” into DOM.
2. element nodes – HTML-tags, the tree building blocks.
3. text nodes – contain text.
4. comments – sometimes we can put information there, it won't be shown, but JS can read it from the DOM.

2. DOM: DOM tree

Summary

An HTML/XML document is represented inside the browser as the DOM tree.

- Tags become element nodes and form the structure.
- Text becomes text nodes.
- ...etc, everything in HTML has its place in DOM, even comments.

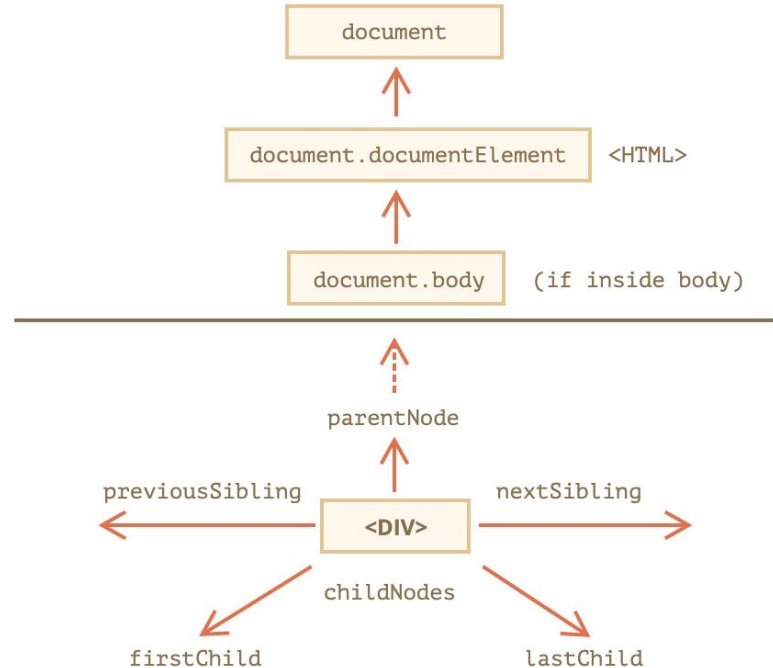
We can use developer tools to inspect DOM and modify it manually.

3. DOM: Walking the DOM - Topmost Element Nodes

The DOM allows us to do anything with elements and their contents, but first we need to reach the corresponding DOM object.

All operations on the DOM start with the `document` object. That's the main "entry point" to DOM. From it we can access any node.

Here's a picture of links that allow for travel between DOM nodes:



3. DOM: Walking the DOM - Topmost Element Nodes

On top: documentElement and body

The topmost tree nodes are available directly as `document` properties:

`<html> = document.documentElement`

The topmost document node is `document.documentElement`. That's the DOM node of the `<html>` tag.

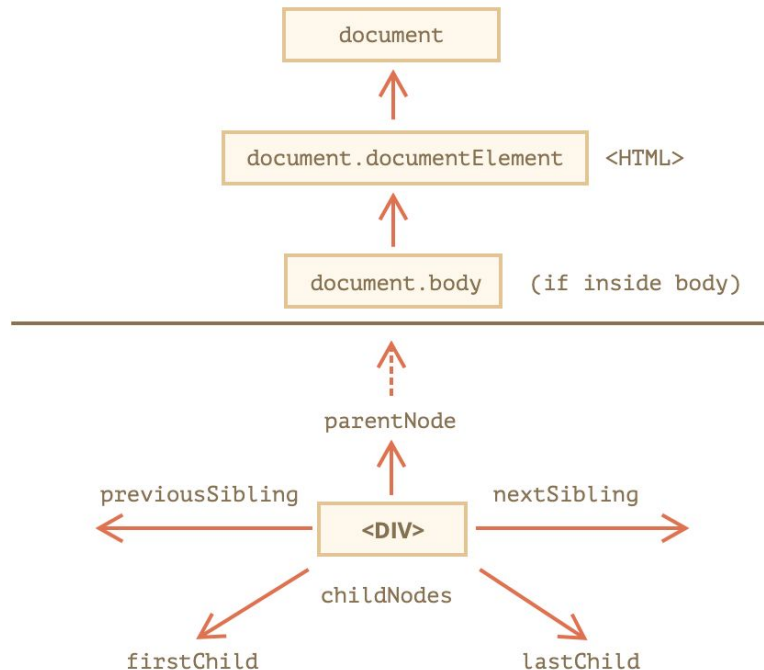
`<body> = document.body`

Another widely used DOM node is the `<body>` element – `document.body`.

`<head> = document.head`

The `<head>` tag is available as `document.head`.

Here's a picture of links that allow for travel between DOM nodes:



3. DOM: Walking the DOM - Node navigation

Children: `childNodes`, `firstChild`, `lastChild`

There are two terms that we'll use from now on:

- **Child nodes (or children)** – elements that are direct children. In other words, they are nested exactly in the given one. For instance, `<head>` and `<body>` are children of `<html>` element.
- **Descendants** – all elements that are nested in the given one, including children, their children and so on.

For instance, here `<body>` has children `<div>` and `<ul>`

...And descendants of `<body>` are not only direct children `<div>`, `<ul>` but also more deeply nested elements, such as `<li>` (a child of `<ul>`) and `<b>` (a child of `<li>`) – the entire subtree.

```
1  <html>
2  <body>
3    <div>Begin</div>
4
5    <ul>
6      <li>
7        <b>Information</b>
8      </li>
9    </ul>
10 </body>
11 </html>
```


3. DOM: Walking the DOM - Node navigation

The screenshot displays a web browser's developer console. On the left, the 'index.html' file is open, showing the following HTML structure:

```
1 <!doctype html>
2 <html>
3 <body>
4   <div>Begin</div>
5
6   <ul>
7     <li>Information</li>
8   </ul>
9
10  <div>End</div>
11
12  <script>
13    for (let node of document.body.childNodes) {
14      console.log( node );
15    }
16  </script>
17 </body>
18 </html>
```

On the right, the 'Console' tab is active, showing the output of the script. The console displays a list of DOM nodes, including text nodes and element nodes, with their corresponding indices (e.g., (index):14). The console also shows the URL: `-url:https://server.ett`.

```
> document.body.childNodes
< NodeList []
```

NodeList Properties `firstChild` and `lastChild` give fast access to the first and last children.

3. DOM: Walking the DOM - Node navigation

NodeList and HTMLCollection, look like arrays. But aren't, they are a *collection* object– a special array-like iterable object.

There are two important consequences:

1. We can use `for...of` to iterate over it:

```
1 for (let node of document.body.childNodes) {  
2   alert(node); // shows all nodes from the collection  
3 }
```

2. Array methods won't work, because it's not an array.

However, we can use `Array.from` or the spread operator to create a “real” array from the collection.

```
1 alert( Array.from(document.body.childNodes).filter ); // function
```

3. DOM: Walking the DOM - Node navigation



DOM collections are read-only

DOM collections, and even more – *all* navigation properties listed in this chapter are read-only.

We can't replace a child by something else by assigning `childNodes[i] = ...`.

Changing DOM needs other methods. We will see them in the next chapter.



DOM collections are live

Almost all DOM collections with minor exceptions are *live*. In other words, they reflect the current state of DOM.

If we keep a reference to `elem.childNodes`, and add/remove nodes into DOM, then they appear in the collection automatically.

3. DOM: Walking the DOM - Node navigation

Siblings and the parent

Siblings are nodes that are children of the same parent. For instance, here `<head>` and `<body>` are siblings:

```
1 <html>
2   <head>...</head><body>...</body>
3 </html>
```

- `<body>` is said to be the “next” or “right” sibling of `<head>`,
- `<head>` is said to be the “previous” or “left” sibling of `<body>`.

The next sibling is in `nextSibling` property, and the previous one – in `previousSibling`.

The parent is available as `parentNode`.

3. DOM: Walking the DOM - Node navigation

Siblings and the parent

For example:

```
1 // parent of <body> is <html>
2 alert( document.body.parentNode === document.documentElement ); // true
3
4 // after <head> goes <body>
5 alert( document.head.nextSibling ); // HTMLBodyElement
6
7 // before <body> goes <head>
8 alert( document.body.previousSibling ); // HTMLHeadElement
```

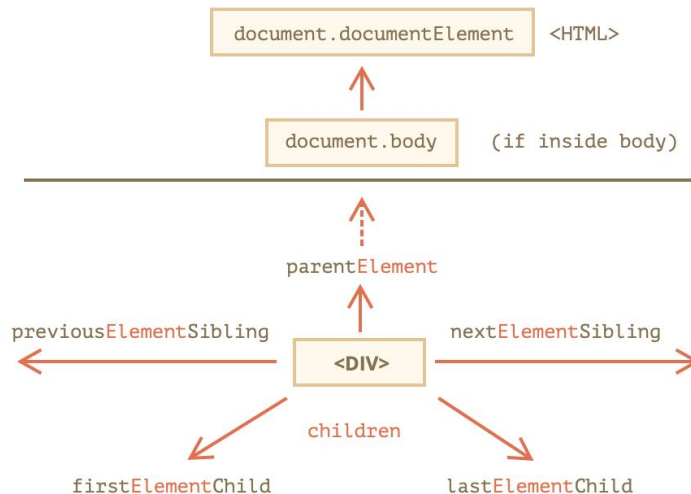
3. DOM: Walking the DOM - Element-only navigation

Element-only navigation

Navigation properties listed above refer to *all* nodes.
For instance, in `childNodes` it includes:

- text nodes,
- element nodes,
- comment nodes.

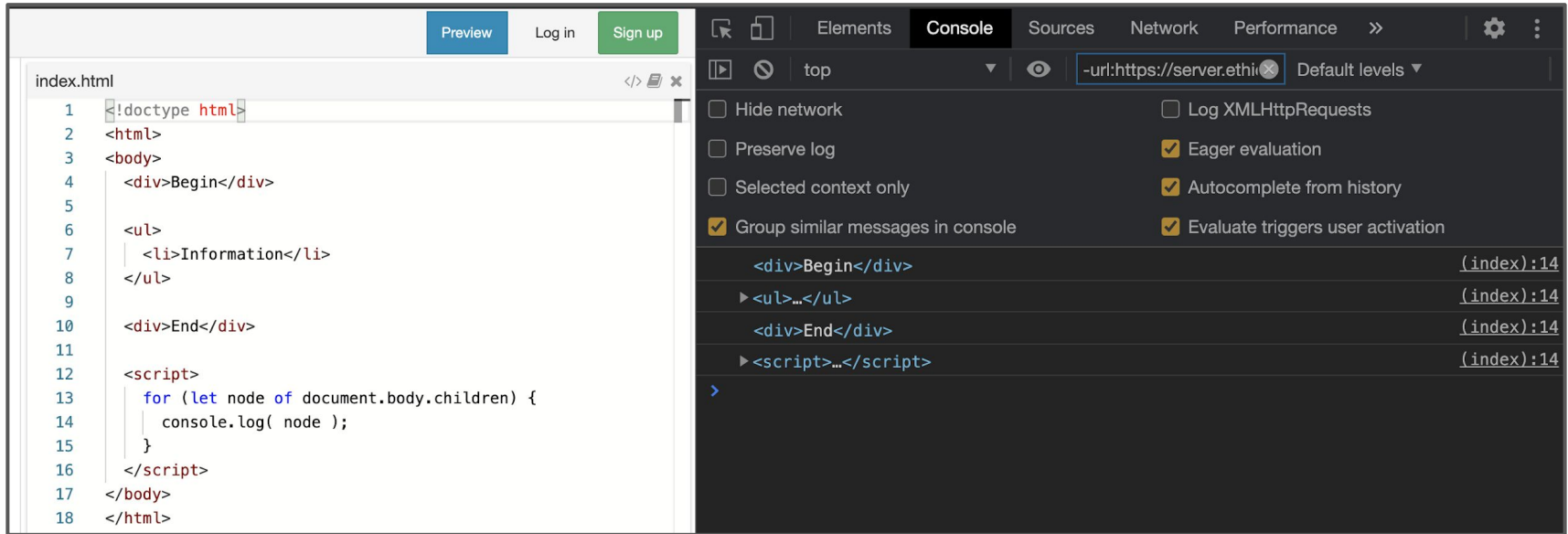
Typically you don't need text or comment nodes.
Just the element nodes that represent HTML tags.



The links are similar to those given above, just with `Element` word inside:

- `children` – only those children that are element nodes.
- `firstElementChild`, `lastElementChild` – first and last element children.
- `previousElementSibling`, `nextElementSibling` – neighbor elements.
- `parentElement` – parent element.

3. DOM: Walking the DOM - Element-only navigation



```
> document.body.children
< ▶ HTMLCollection []
```

HTMLCollection Properties `firstElementChild` and `lastElementChild` give fast access to the first and last children.

3. DOM: Walking the DOM

Summary

Given a DOM node, we can go to its immediate neighbors using navigation properties.

There are two main sets of them:

- For all nodes:
 - `parentNode`,
 - `childNodes`, (`NodeList`)
 - `firstChild`, `lastChild`,
 - `previousSibling`, `nextSibling`.
- For element nodes only:
 - `parentElement`,
 - `children`, (`HTMLCollection`)
 - `firstElementChild`, `lastElementChild`,
 - `previousElementSibling`, `nextElementSibling`.

Note: Some types of DOM elements, e.g. tables, provide additional properties and collections to access their content.

4. DOM: Searching: getElement*

There are many DOM searching methods.

document.getElementById()

If an element has the `id` attribute, we can get the element using the method `document.getElementById(id)`.

```
1 <div id="elem">
2   <div id="elem-content">Element</div>
3 </div>
4
5 <script>
6   // get the element
7   let elem = document.getElementById('elem');
8
9   // make its background red
10  elem.style.background = 'red';
11 </script>
```

4. DOM: Searching: getElement*

The `id` must be unique

The `id` must be unique. There can be only one element in the document with the given `id`.

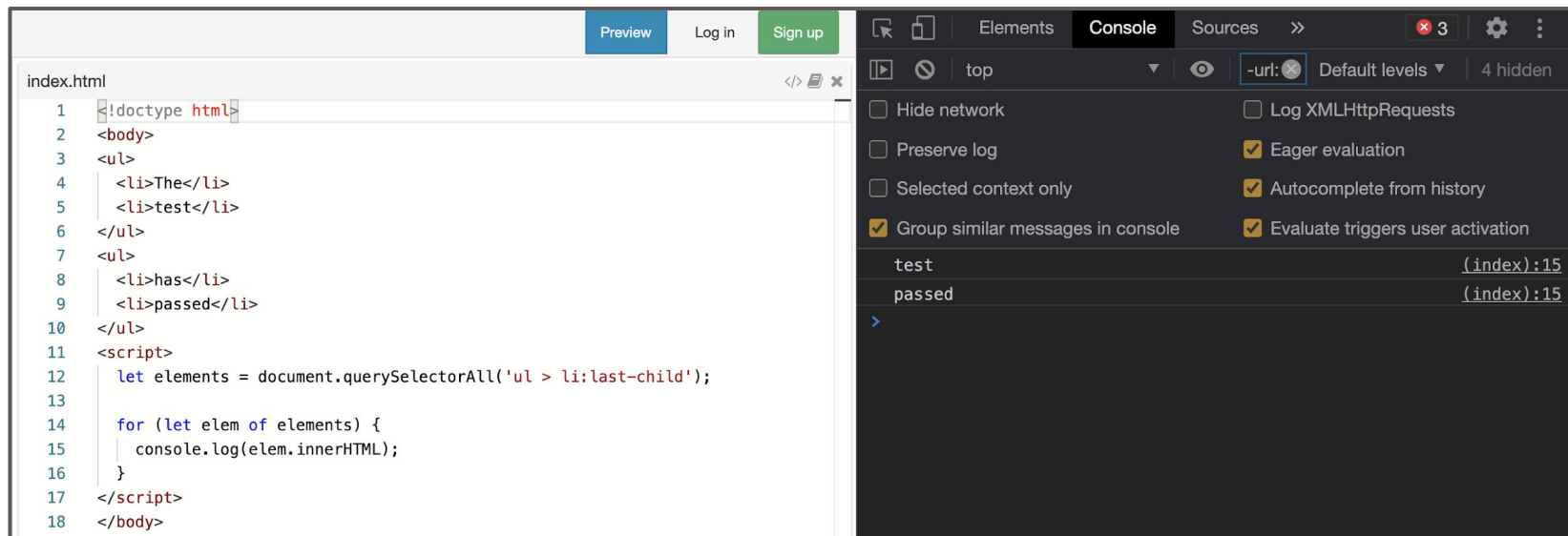
If there are multiple elements with the same `id`, then the behavior of methods that use it is unpredictable, e.g. `document.getElementById` may return any of such elements at random. So please stick to the rule and keep `id` unique.

4. DOM: Searching: querySelector*

querySelectorAll

`element.querySelectorAll(css)` returns all elements inside `element` matching the given any CSS selector.

Example: Here we look for all `<li>` elements that are last children:



The screenshot displays a web browser interface with two main panels. The left panel shows the source code of a file named `index.html`. The code defines an HTML document with two `ul` elements. The first `ul` contains `The` and `test` as list items. The second `ul` contains `has` and `passed`. A JavaScript script is included that uses `document.querySelectorAll('ul > li:last-child')` to select the last child of each `ul` and logs their innerHTML to the console. The right panel shows the browser's developer tools, specifically the Console tab. It displays two log messages: `test` and `passed`, both originating from `(index):15`. The console also shows settings for log levels (Default levels) and a list of 4 hidden messages.

```
index.html
1 <!doctype html>
2 <body>
3 <ul>
4   <li>The</li>
5   <li>test</li>
6 </ul>
7 <ul>
8   <li>has</li>
9   <li>passed</li>
10 </ul>
11 <script>
12   let elements = document.querySelectorAll('ul > li:last-child');
13
14   for (let elem of elements) {
15     console.log(elem.innerHTML);
16   }
17 </script>
18 </body>
```

Console

top

Hide network

Log XMLHttpRequests

Preserve log

Eager evaluation

Selected context only

Autocomplete from history

Group similar messages in console

Evaluate triggers user activation

test (index):15

passed (index):15

4. DOM: Searching: `querySelector`*

`querySelector`

The call to `element.querySelector(css)` returns the first element for the given CSS selector.

In other words, the result is the same as `element.querySelectorAll(css)[0]`, but the latter is looking for *all* elements and picking one, while `elem.querySelector` just looks for one. So it's faster and also shorter to write.

4. DOM: Searching: matches

matches

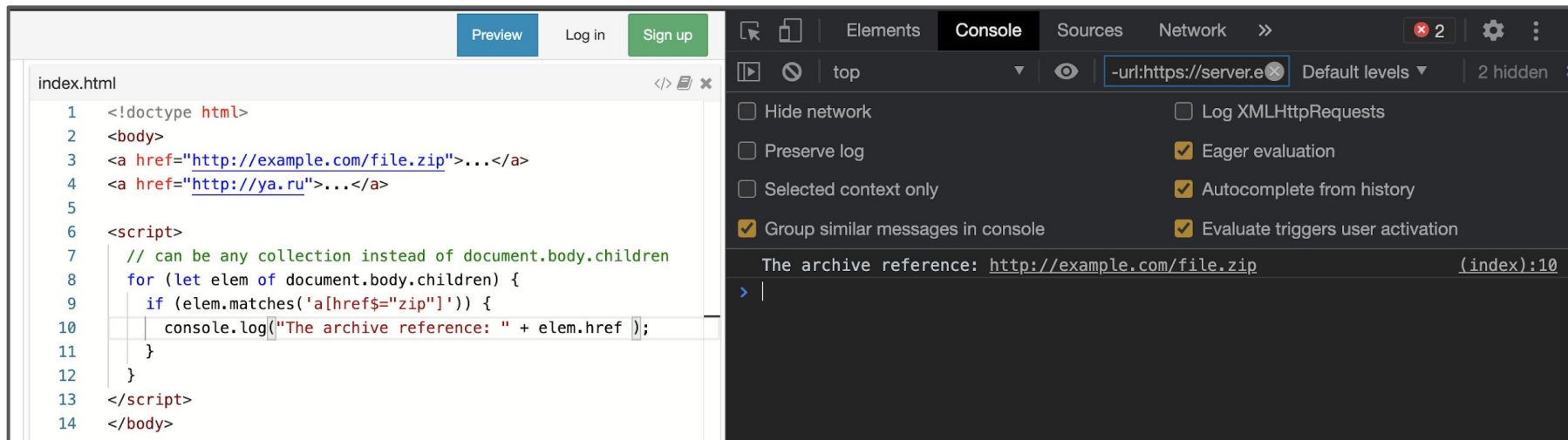
Previous methods were searching the DOM.

The `element.matches(css)` does not look for anything, it merely checks if `element` matches the given CSS-selector. It returns `true` or `false`.

The method comes in handy when we are iterating over elements (like in an array or something) and trying to filter out those that interest us.

4. DOM: Searching: matches

matches (Example)



The screenshot displays a web browser interface with a code editor on the left and the console on the right.

Code Editor (index.html):

```
1 <!doctype html>
2 <body>
3 <a href="http://example.com/file.zip">...</a>
4 <a href="http://ya.ru">...</a>
5
6 <script>
7   // can be any collection instead of document.body.children
8   for (let elem of document.body.children) {
9     if (elem.matches('a[href$=".zip"]')) {
10      console.log("The archive reference: " + elem.href );
11    }
12  }
13 </script>
14 </body>
```

Console:

The console shows the output of the script execution:

```
The archive reference: http://example.com/file.zip (index):10
```

The console also displays various settings and filters, including a filter for the URL `-url:https://server.e`.

4. DOM: Searching: closest

closest

Ancestors of an element are: parent, the parent of parent, its parent and so on. The ancestors together form the chain of parents from the element to the top.

The method `elem.closest(css)` looks for the nearest ancestor that matches the CSS-selector. The `elem` itself is also included in the search.

In other words, the method `closest` goes up from the element and checks each of parents. If it matches the selector, then the search stops, and the ancestor is returned.

4. DOM: Searching: closest

closest (Example)

index.html

```
1 <!doctype html>
2 <body>
3 <h1>Contents</h1>
4
5 <div class="contents">
6   <ul class="book">
7     <li class="chapter">Chapter 1</li>
8     <li class="chapter">Chapter 1</li>
9   </ul>
10 </div>
11
12 <script>
13   let chapter = document.querySelector('.chapter'); // LI
14   console.log(chapter.closest('.book')); // UL
15   console.log(chapter.closest('.contents')); // DIV
16   console.log(chapter.closest('h1')); // null (because h1 is not an ancestor)
17 </script>
18 </body>
```

Preview Log in Sign up

Elements Console Sources Network >>

top -url:https://server.e... Default levels

☐ Hide network

☐ Log XMLHttpRequests

☐ Preserve log

☐ Selected context only

☒ Group similar messages in console

☐ Eager evaluation

☒ Autocomplete from history

☒ Evaluate triggers user activation

> <ul class="book">... (index):14

> <div class="contents">...</div> (index):15

null (index):16

>

4. DOM: Searching: `getElementsBy*`

`getElementsBy*`

There are also other methods to look for nodes by a tag, class, etc.

Today, they are mostly history, as `querySelector` is more powerful and shorter to write.

So here we cover them mainly for completeness, while you can still find them in the old scripts.

- `elem.getElementsByTagName(tag)` looks for elements with the given tag and returns the collection of them. The tag parameter can also be a star `"*"` for “any tags”.
- `elem.getElementsByClassName(className)` returns elements that have the given CSS class.
- `document.getElementsByName(name)` returns elements with the given name attribute, document-wide. Very rarely used.

4. DOM: Searching: `getElementsByTagName`*



It returns a collection, not an element!

Another widespread novice mistake is to write:

```
1 // doesn't work
2 document.getElementsByTagName('input').value = 5;
```

That won't work, because it takes a *collection* of inputs and assigns the value to it rather than to elements inside it.

We should either iterate over the collection or get an element by its index, and then assign, like this:

```
1 // should work (if there's an input)
2 document.getElementsByTagName('input')[0].value = 5;
```

4. DOM: Searching: Live Collections vs Static Collections

All methods `"getElementsBy*"` return a *live* collection. Such collections always reflect the current state of the document and “auto-update” when it changes.

In contrast, `querySelectorAll` returns a *static* collection. It's like a fixed array of elements.

4. DOM: Searching: Summary

There are 6 main methods to search for nodes in DOM:

Method	Searches by...	Can call on an element?	Live?
<code>querySelector</code>	CSS-selector	✓	-
<code>querySelectorAll</code>	CSS-selector	✓	-
<code>getElementById</code>	<code>id</code>	-	-
<code>getElementsByName</code>	<code>name</code>	-	✓
<code>getElementsByTagName</code>	tag or <code>'*'</code>	✓	✓
<code>getElementsByClassName</code>	class	✓	✓

5. DOM: Node properties: Intro

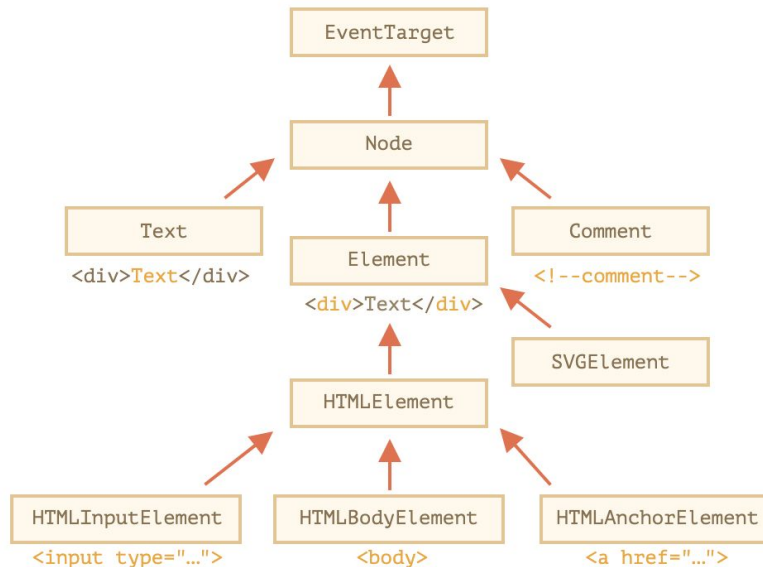
DOM node classes

Different DOM nodes may have different properties.

For instance, an element node corresponding to tag `<a>` has link-related properties, and the one corresponding to `<input>` has input-related properties and so on. Text nodes are not the same as element nodes but share some properties and methods, as all classes of DOM nodes form a single hierarchy.

Each DOM node belongs to the corresponding built-in class.

The root of the hierarchy is `EventTarget`, that is inherited by `Node`, and other DOM nodes inherit from it.



5. DOM: Node properties: Intro

The classes are:

- **EventTarget** – is the root “abstract” class. Objects of that class are never created. It serves as a base, so that all DOM nodes support so-called “events”, we’ll study them later.
- **Node** – is also an “abstract” class, serving as a base for DOM nodes. It provides the core tree functionality: `parentNode`, `nextSibling`, `childNodes` and so on (they are getters). Objects of `Node` class are never created. But there are concrete node classes that inherit from it, namely: `Text` for text nodes, `Element` for element nodes and more exotic ones like `Comment` for comment nodes.
- **Element** – is a base class for DOM elements. It provides element-level navigation like `nextElementSibling`, `children` and searching methods like `getElementsByTagName`, `querySelector`. A browser supports not only HTML, but also XML and SVG. The `Element` class serves as a base for more specific classes: `SVGElement`, `XMLElement` and `HTMLElement`.
- **HTMLElement** – is finally the basic class for all HTML elements. It is inherited by concrete HTML elements:
 - **HTMLInputElement** – the class for `<input>` elements,
 - **HTMLBodyElement** – the class for `<body>` elements,
 - **HTMLAnchorElement** – the class for `<a>` elements,
 - ...and so on, each tag has its own class that may provide specific properties and methods.

5. DOM: Node properties: Intro

The full set of properties and methods of a given node comes as the result of the inheritance.

For example, let's consider the DOM object for an `<input>` element. It belongs to `HTMLInputElement` class.

It gets properties and methods as a superposition of (listed in inheritance order):

- `HTMLInputElement` – this class provides input-specific properties,
- `HTMLElement` – it provides common HTML element methods (and getters/setters),
- `Element` – provides generic element methods,
- `Node` – provides common DOM node properties,
- `EventTarget` – gives the support for events (to be covered later),
- ...and finally it inherits from `Object`, so “plain object” methods like `hasOwnProperty` are also available.

5. DOM: Node properties: type

The “nodeType” property

The `nodeType` property provides one more, “old-fashioned” way to get the “type” of a DOM node.

It has a numeric value:

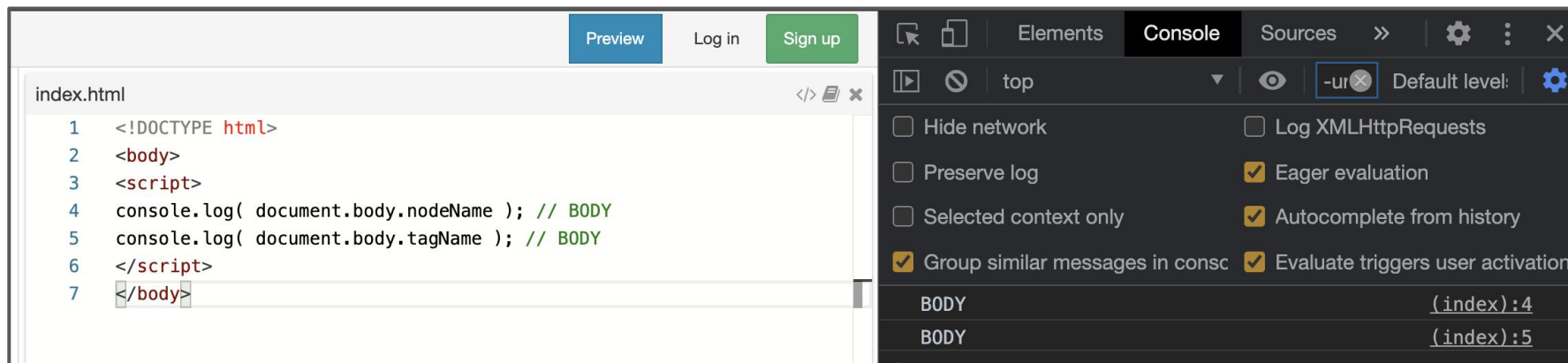
- `elem.nodeType == 1` for element nodes,
- `elem.nodeType == 3` for text nodes,
- `elem.nodeType == 9` for the document object,

5. DOM: Node properties: tag

Tag: nodeName and tagName

Given a DOM node, we can read its tag name from `nodeName` or `tagName` properties:

For instance:



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays two log messages: 'BODY' at index 4 and 'BODY' at index 5. The code in the editor on the left is as follows:

```
index.html
1 <!DOCTYPE html>
2 <body>
3 <script>
4 console.log( document.body.nodeName ); // BODY
5 console.log( document.body.tagName ); // BODY
6 </script>
7 </body>
```

The console interface includes a toolbar with icons for running, pausing, and clearing logs, a dropdown menu for selecting the log level (currently set to 'Error'), and a 'Default level' button. The console also has checkboxes for various settings: 'Hide network', 'Log XMLHttpRequests', 'Preserve log', 'Eager evaluation', 'Selected context only', 'Autocomplete from history', 'Group similar messages in console', and 'Evaluate triggers user activation'. The log messages are displayed in a table with columns for the message content and the index of the log entry.

Message	Index
BODY	(index):4
BODY	(index):5

5. DOM: Node properties: innerHTML

innerHTML: the contents

The `innerHTML` property allows to get the HTML inside the element as a string.

We can also modify it. So it's one of the most powerful ways to change the page.

The image shows a web browser interface with a code editor on the left, a preview window in the middle, and a developer console on the right.

Code Editor (index.html):

```
1 <!doctype html>
2 <body>
3   <p>A paragraph</p>
4   <div>A div</div>
5
6   <script>
7     console.log( document.body.innerHTML ); // original contents
8     document.body.innerHTML = 'The new BODY!'; // replace it
9   </script>
10
11 </body>
```

Preview:

The new BODY!

Developer Console:

- Hide network
- Preserve log
- Selected context only
- Group similar messages in
- Log XMLHttpRequests
- Eager evaluation
- Autocomplete from history
- Evaluate triggers user activity

Console was cleared VM2430:1

< undefined

(index):7

```
<p>A paragraph</p>
<div>A div</div>

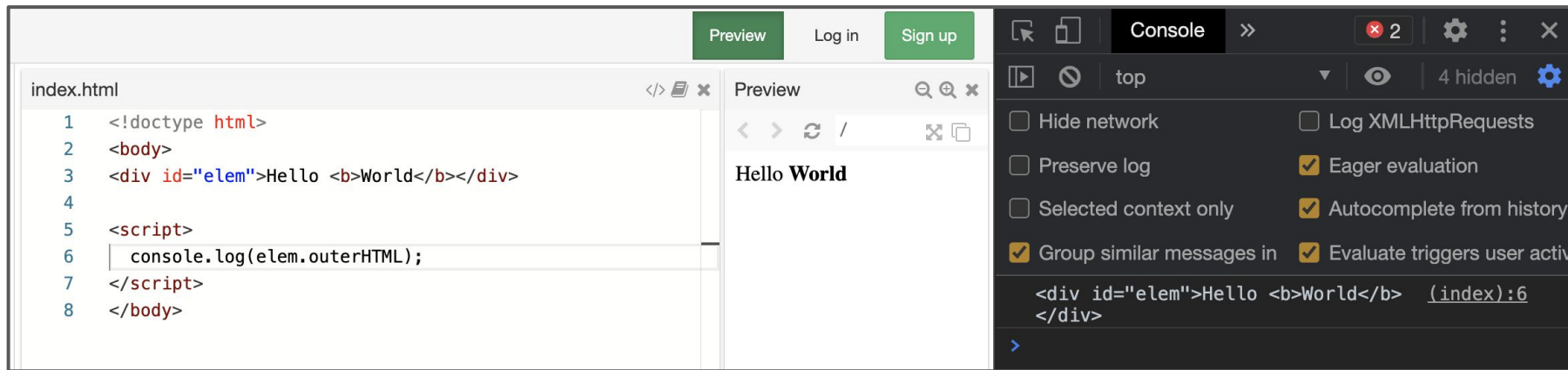
<script>
  console.log( document.body.innerHTML );
  // original contents
  document.body.innerHTML = 'The new
  BODY!'; // replace it
</script>
```

5. DOM: Node properties: outerHTML

outerHTML: full HTML of the element

The `outerHTML` property contains the full HTML of the element. That's like `innerHTML` plus the element itself.

Here's an example:

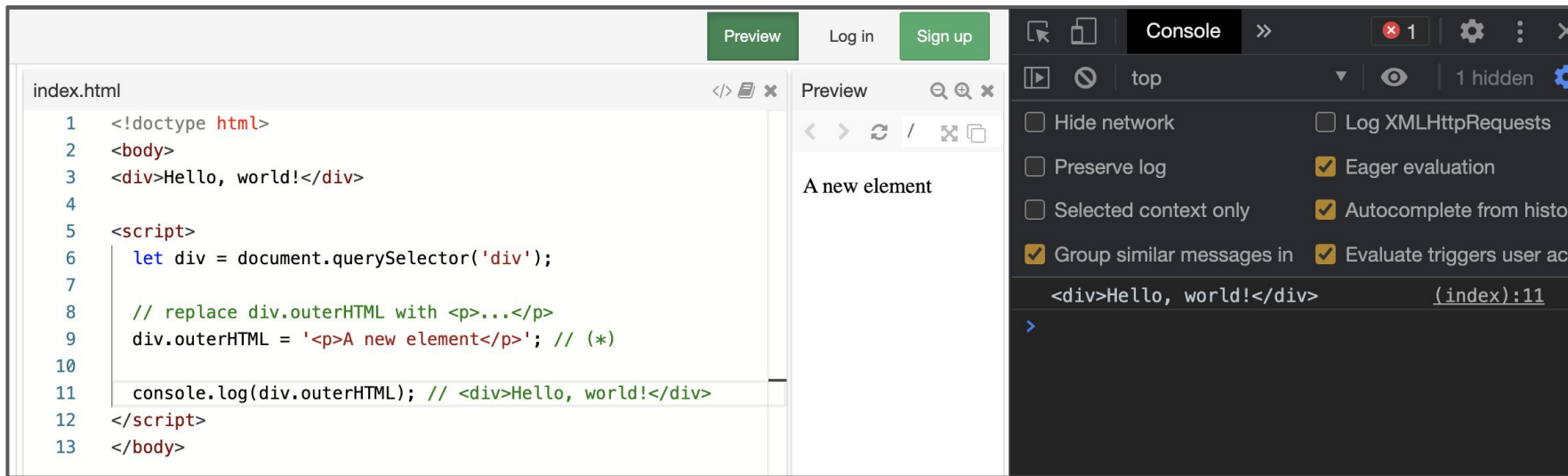


5. DOM: Node properties: outerHTML

Beware:

Unlike `innerHTML`, writing to `outerHTML` does not change the element. Instead, it replaces it in the DOM.

Consider the example: (Explanation: `div` becomes an obsolete reference that no longer exists in the DOM)



The screenshot displays a web development environment with three main panels:

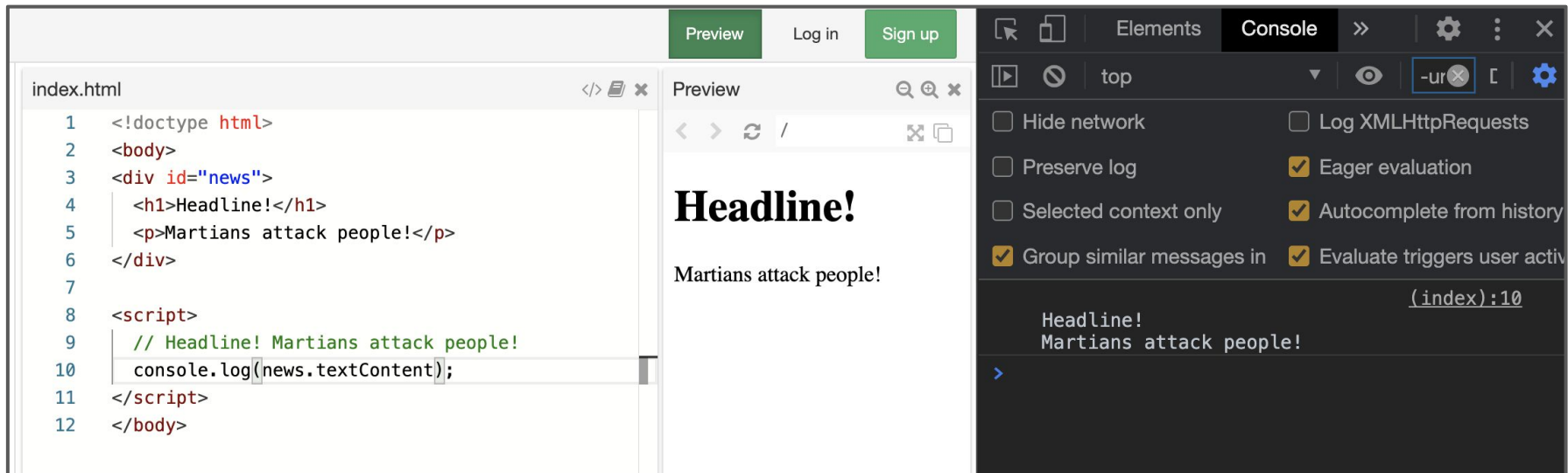
- Code Editor:** Shows the `index.html` file with the following content:

```
1 <!doctype html>
2 <body>
3 <div>Hello, world!</div>
4
5 <script>
6   let div = document.querySelector('div');
7
8   // replace div.outerHTML with <p>...</p>
9   div.outerHTML = '<p>A new element</p>'; // (*)
10
11   console.log(div.outerHTML); // <div>Hello, world!</div>
12 </script>
13 </body>
```
- Preview:** Displays the rendered output, showing "A new element".
- Console:** Shows a log message: `<div>Hello, world!</div>` from `(index):11`. The console also includes settings for network, log, and evaluation options.

5. DOM: Node properties: textContent

textContent: pure text

The `textContent` provides access to the text inside the element: only text, minus all `<tags>`.



The screenshot displays a web browser's developer console with three main sections:

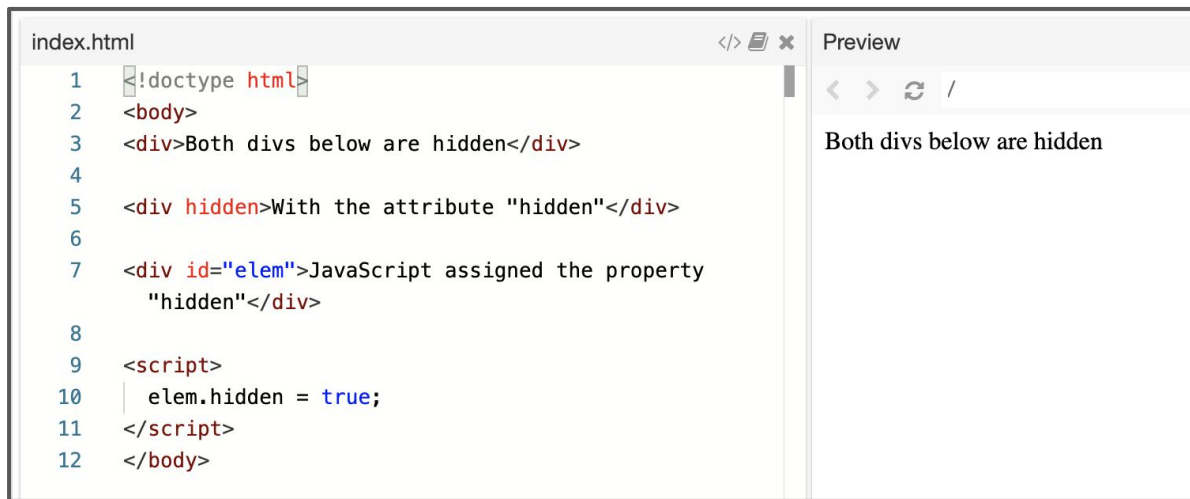
- Code Editor:** Shows the HTML structure of `index.html`. It includes a `<div id="news">` containing a `<h1>Headline!</h1>` and a `<p>Martians attack people!</p>`. A `<script>` block contains the code `console.log(news.textContent);`.
- Preview:** A visual representation of the rendered HTML, showing the text "Headline!" in a large, bold font, followed by "Martians attack people!" in a smaller font.
- Console:** Displays the output of the JavaScript code. It shows two log entries: "Headline!" and "Martians attack people!". The first log entry is highlighted, and the console shows the source as `(index):10`.

5. DOM: Node properties: hidden

The “hidden” property

The “hidden” attribute and the DOM property specifies whether the element is visible or not.

We can use it in HTML or assign it using JavaScript, like this:



```
index.html
1 <!doctype html>
2 <body>
3 <div>Both divs below are hidden</div>
4
5 <div hidden>With the attribute "hidden"</div>
6
7 <div id="elem">JavaScript assigned the property
  "hidden"</div>
8
9 <script>
10   elem.hidden = true;
11 </script>
12 </body>
```

Preview

Both divs below are hidden

5. DOM: Node properties: More Properties

More properties

DOM elements also have additional properties, in particular those that depend on the class:

- `value` – the value for `<input>`, `<select>` and `<textarea>` (`HTMLInputElement`, `HTMLSelectElement`...).
- `href` – the “href” for `<a href=“...”>` (`HTMLAnchorElement`).
- `src` – the “src” for `<img src=“...”>` (`HTMLImageElement`).
- `id` – the value of “id” attribute, for all elements (`HTMLElement`).
- ...and much more...

If you want to know the full list of supported properties for a given class – you can always output the element using `console.dir(elem)` and read the properties. Or explore “DOM properties” in the Elements tab of the browser developer tools.

6. DOM: Attributes and properties

TBD

7. DOM: Modifying the document

DOM modification is the key to creating “interactive” pages.

Here we'll see how to create new elements “on the fly” and modify the existing page content

7. DOM: Modifying the document

Example: show a message

Let's demonstrate using an example. We'll add a message on the page that looks nicer than `alert`.

Here's how it will look:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<div class="alert">
  <strong>Hi there!</strong> You've read an important message.
</div>
```

Hi there! You've read an important message.

7. DOM: Modifying the document

Creating an element

To create DOM nodes, there are two methods:

`document.createElement(tag)`

Creates a new *element node* with the given tag:

```
let div = document.createElement('div');
```

`document.createTextNode(text)`

Creates a new *text node* with the given text:

```
let textNode = document.createTextNode('Here I am');
```

Most of the time we need to create element nodes, such as the `div` for the message.

7. DOM: Modifying the document

Creating the message

Creating the message div takes 3 steps:

```
// 1. Create <div> element
let div = document.createElement('div');

// 2. Set its class to "alert"
div.className = "alert";

// 3. Fill it with the content
div.innerHTML = "<strong>Hi there!</strong> You've read an important message.";
```

We've created the element. But as of now it's only in a variable named `div`, not in the page yet. So we can't see it.

7. DOM: Modifying the document

Insertion methods

To display the `div`, it must be inserted into `document`.

There's the insertion method: `append`

Here's the full code:

For example: the `<body>` element, via `document.body`.

For example: `document.body.append(div)`.

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>Hi there!</strong> You've read an important message.";

  document.body.append(div);
</script>
```

Here we called `append` on `document.body`, but we can call `append` method on any other element, to put another element into it.

For instance, we can append something to `<div>` by calling `div.append(anotherElement)`.

7. DOM: Modifying the document

Insertion methods

Here are more insertion methods, they specify different places where to insert:

- `node.append(...nodes or strings)` – append nodes or strings *at the end* of `node`,
- `node.prepend(...nodes or strings)` – insert nodes or strings *at the beginning* of `node`,
- `node.before(...nodes or strings)` -- insert nodes or strings *before* `node`,
- `node.after(...nodes or strings)` -- insert nodes or strings *after* `node`,
- `node.replaceWith(...nodes or strings)` -- replaces `node` with the given nodes or strings.

Arguments of these methods are an arbitrary list of DOM nodes to insert, or text strings (that become text nodes automatically).

7. DOM: Modifying the document

Insertion methods

Here's an example of using these methods to add items to a list and the text before/after it:

```
<ol id="ol">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  ol.before('before'); // insert string "before" before <ol>
  ol.after('after'); // insert string "after" after <ol>

  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // insert liFirst at the beginning of <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // insert liLast at the end of <ol>
</script>
```

before

1. prepend
2. 0
3. 1
4. 2
5. append

after

7. DOM: Modifying the document

insertAdjacentHTML/Text/Element

For inserting html via JS, methods: `elem.insertAdjacentHTML(where, html)`.

The first parameter is a code word, specifying where to insert relative to `elem`. Must be one of the following:

- `"beforebegin"` – insert `html` immediately before `elem`,
- `"afterbegin"` – insert `html` into `elem`, at the beginning,
- `"beforeend"` – insert `html` into `elem`, at the end,
- `"afterend"` – insert `html` immediately after `elem`.

The second parameter is an HTML string, that is inserted “as HTML”.

7. DOM: Modifying the document

insertAdjacentHTML/Text/Element

The second parameter is an HTML string, that is inserted "as HTML".

For instance:

```
<div id="div">Here</div>
<script>
  div.insertAdjacentHTML('beforebegin', '<p>Hello</p>');
  div.insertAdjacentHTML('afterend', '<p>Bye</p>');
</script>
```

...Would lead to:

```
<p>Hello</p>
<div id="div">Here</div>
<p>Bye</p>
```

7. DOM: Modifying the document

Node removal

To remove a node, there's a method `node.remove()`.

Let's make our message disappear after a second:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>Hi there!</strong> You've read an important message.";

  document.body.append(div);
  setTimeout(() => div.remove(), 1000);
</script>
```

7. DOM: Modifying the document

Node removal

Please note: if we want to *move* an element to another place – there's no need to remove it from the old one.

All insertion methods automatically remove the node from the old place.

For instance, let's swap elements:

```
<div id="first">First</div>
<div id="second">Second</div>
<script>
  // no need to call remove
  second.after(first); // take #second and after it insert #first
</script>
```

Second
First

7. DOM: Modifying the document

Cloning nodes: cloneNode

You may *clone* existing elements & modify them (*if needed*).

With big elements, cloning may be faster and simpler.

- `elem.cloneNode(true)` creates a “deep” clone of the element – with all attributes and subelements.
- `elem.cloneNode(false)`, then the clone is made without child elements.

An example of copying the message:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<div class="alert" id="div">
  <strong>Hi there!</strong> You've read an important message.
</div>

<script>
  let div2 = div.cloneNode(true); // clone the message
  div2.querySelector('strong').innerHTML = 'Bye there!'; // change the clone

  div.after(div2); // show the clone after the existing div
</script>
```

Hi there! You've read an important message.

Bye there! You've read an important message.

7. DOM: Modifying the document

Summary

- Methods to create new nodes:
 - `document.createElement(tag)` – creates an element with the given tag,
 - `document.createTextNode(value)` – creates a text node (rarely used),
 - `elem.cloneNode(deep)` – clones the element, if `deep==true` then with all descendants.
- Insertion and removal:
 - `node.append(...nodes or strings)` – insert into node, at the end,
 - `node.prepend(...nodes or strings)` – insert into node, at the beginning,
 - `node.before(...nodes or strings)` -- insert right before node,
 - `node.after(...nodes or strings)` -- insert right after node,
 - `node.replaceWith(...nodes or strings)` -- replace node.
 - `node.remove()` -- remove the node.
- Text strings are inserted “as text”.
- All these methods return node.
- Given some HTML in `html`, `elem.insertAdjacentHTML(whence, html)` inserts it depending on the value of `whence`:
 - `"beforebegin"` – insert `html` right before `elem`,
 - `"afterbegin"` – insert `html` into `elem`, at the beginning,
 - `"beforeend"` – insert `html` into `elem`, at the end,
 - `"afterend"` – insert `html` right after `elem`.

8. DOM: Styles & Classes

Intro

Before we get into JavaScript's ways of dealing with styles and classes – here's an important rule.

There are generally two ways to style an element:

1. Create a class in CSS and add it: `<div class="...">`
2. Write properties directly into `style`: `<div style="...">`.

JavaScript can modify both classes and `style` properties.

We should always prefer CSS classes to `style`. The latter should only be used if classes “can't handle it”.

8. DOM: Styles & Classes

Intro

For example, `style` is acceptable if we calculate coordinates of an element dynamically and want to set them from JavaScript, like this:

```
1 let top = /* complex calculations */;  
2 let left = /* complex calculations */;  
3  
4 elem.style.left = left; // e.g '123px', calculated at run-time  
5 elem.style.top = top; // e.g '456px'
```

For other cases, like making the text red, adding a background icon – describe that in CSS and then add the class (JavaScript can do that). That's more flexible and easier to support.

8. DOM: Styles & Classes

className and classList

Changing a class is one of the most often used actions in scripts.

In JavaScript: `"class"` is a reserved word, thus it can not be used as an object property, like `elem.class`.

So for classes the similar-looking property `"className"` was introduced:

- `elem.className` corresponds to the `"class"` attribute.

8. DOM: Styles & Classes

className and classList

```
1 <body class="main page">
2   <script>
3     alert(document.body.className); // main page
4   </script>
5 </body>
```

If we assign something to `elem.className`, it replaces the whole string of classes. Sometimes that's what we need, but often we want to add/remove a single class.

There's another property for that: `elem.classList`.

The `elem.classList` is a special object with methods to `add/remove/toggle` a single class.

8. DOM: Styles & Classes

className and classList

```
1
2 <body class="main page">
3   <script>
4     // add a class
5     document.body.classList.add('article');
6
7     alert(document.body.className); // main page article
8   </script>
9 </body>
```

So we can operate both on the full class string using `className` or on individual classes using `classList`. What we choose depends on our needs.

Methods of `classList`:

- `elem.classList.add/remove("class")` – adds/removes the class.
- `elem.classList.toggle("class")` – adds the class if it doesn't exist, otherwise removes it.
- `elem.classList.contains("class")` – checks for the given class, returns `true/false`.

8. DOM: Styles & Classes

className and classList

Besides, `classList` is iterable, so we can list all classes with `for...of`, like this:

```
1 <body class="main page">
2   <script>
3     for (let name of document.body.classList) {
4       alert(name); // main, and then page
5     }
6   </script>
7 </body>
```

8. DOM: Styles & Classes

Element style

The property `elem.style` is an object that corresponds to what's written in the `"style"` attribute. Setting `elem.style.width="100px"` works the same as if we had in the attribute `style` a string `width:100px`.

For multi-word property the camelCase is used:

```
1 background-color => elem.style.backgroundColor
2 z-index          => elem.style.zIndex
3 border-left-width => elem.style.borderLeftWidth
```

For instance:

```
document.body.style.backgroundColor = 'green';
```

8. DOM: Styles & Classes

Resetting the style property

Sometimes we want to assign a style property, and later remove it.

For instance, to hide an element, we can set `elem.style.display = "none"`.

Then later we may want to remove the `style.display` as if it were not set. Instead of `delete elem.style.display` we should assign an empty string to it: `elem.style.display = ""`.

```
1 // if we run this code, the <body> will blink
2 document.body.style.display = "none"; // hide
3
4 setTimeout(() => document.body.style.display = "", 1000); // back to normal
```

If we set `style.display` to an empty string, then the browser applies CSS classes and its built-in styles normally, as if there were no such `style.display` property at all.

8. DOM: Styles & Classes

Resetting the style property

Full rewrite with `style.cssText`

Normally, we use `style.*` to assign individual style properties. We can't set the full style like `div.style="color: red; width: 100px"`, because `div.style` is an object, and it's read-only.

To set the full style as a string, there's a special property `style.cssText`:

```
1 <div id="div">Button</div>
2
3 <script>
4   // we can set special style flags like "important" here
5   div.style.cssText=`color: red !important;
6     background-color: yellow;
7     width: 100px;
8     text-align: center;
9 `;
10 </script>
11
12
```

8. DOM: Styles & Classes

Computed styles: `getComputedStyle`

So, modifying a style is easy. But how to *read* it? For instance, we want to know the size, margins, the color of an element. How to do it?

The `style` property operates only on the value of the `"style"` attribute, without any CSS cascade.

So we can't read anything that comes from CSS classes using `elem.style`.

For instance, here `style` doesn't see the margin:

```
1  <head>
2    <style> body { color: red; margin: 5px } </style>
3  </head>
4  <body>
5
6    The red text
7    <script>
8      alert(document.body.style.color); // empty
9      alert(document.body.style.marginTop); // empty
10   </script>
11 </body>
```

8. DOM: Styles & Classes

Computed styles: `getComputedStyle`

```
1 getComputedStyle(element, [pseudo])
```

element

Element to read the value for.

pseudo

A pseudo-element if required, for instance `::before`. An empty string or no argument means the element itself.

The result is an object with styles, like `elem.style`, but now with respect to all CSS classes.

8. DOM: Styles & Classes

Computed styles: getComputedStyle

```
1  <head>
2    <style> body { color: red; margin: 5px } </style>
3  </head>
4  <body>
5
6    <script>
7      let computedStyle = getComputedStyle(document.body);
8
9      // now we can read the margin and the color from it
10
11      alert( computedStyle.marginTop ); // 5px
12      alert( computedStyle.color ); // rgb(255, 0, 0)
13    </script>
14  </body>
15
```

8. DOM: Styles & Classes

Summary

To manage classes, there are two DOM properties:

- `className` – the string value, good to manage the whole set of classes.
- `classList` – the object with methods `add/remove/toggle/contains`, good for individual classes.

To change the styles:

- The `style` property is an object with camelCased styles. Reading and writing to it has the same meaning as modifying individual properties in the `"style"` attribute. To see how to apply `important` and other rare stuff – there's a list of methods at [MDN](#).
- The `style.cssText` property corresponds to the whole `"style"` attribute, the full string of styles.

To read the resolved styles (with respect to all classes, after all CSS is applied and final values are calculated):

- The `getComputedStyle(elem, [pseudo])` returns the style-like object with them. Read-only.

9. DOM: Element size & scrolling

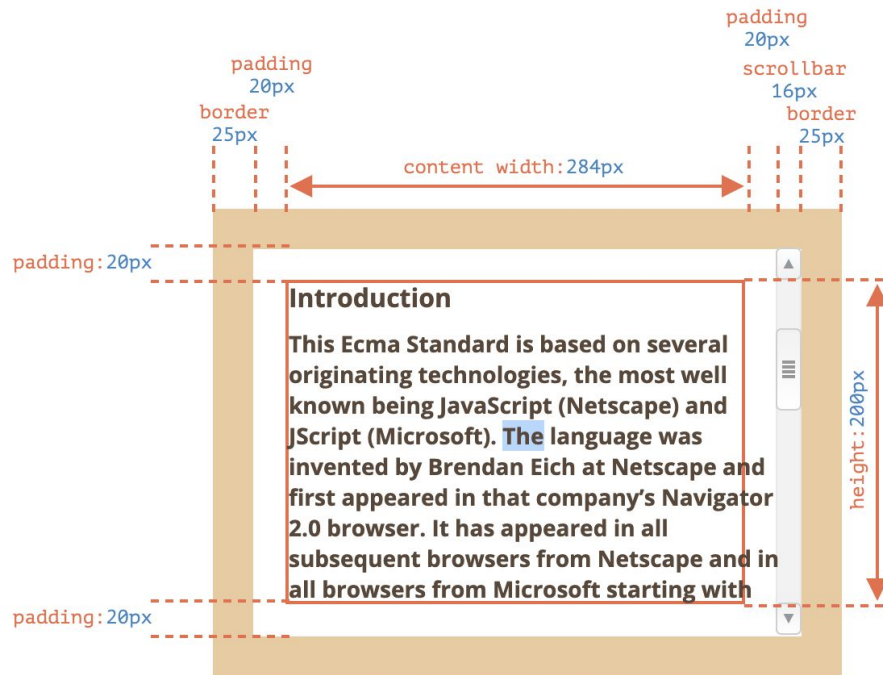
There are many JavaScript properties that allow us to read information about element width, height and other geometry features.

We often need them when moving or positioning elements in JavaScript.

9. DOM: Element size & scrolling

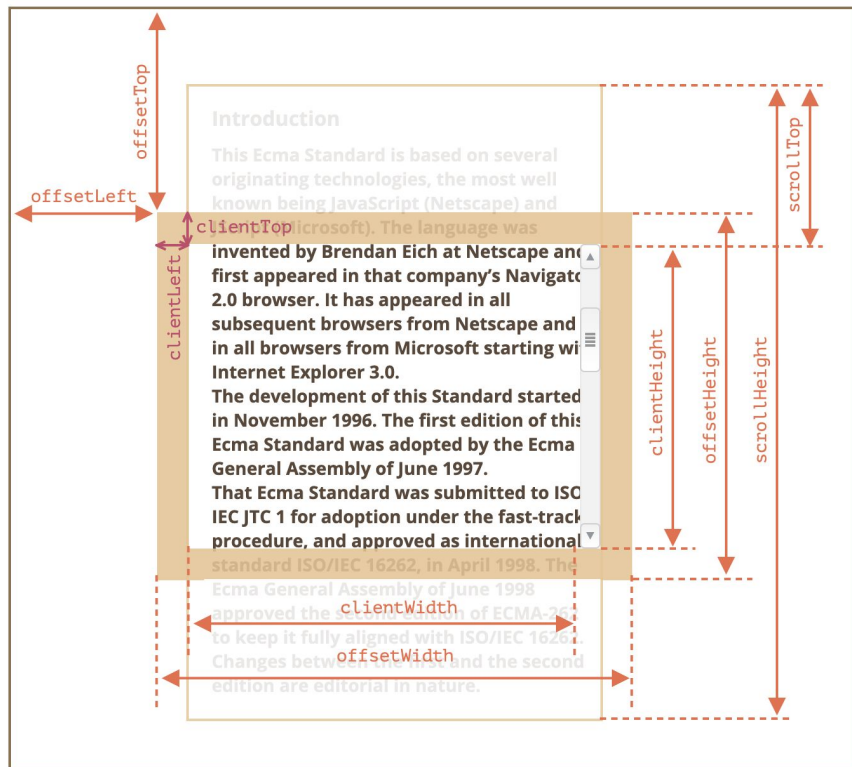
Sample Element

```
1 <div id="example">
2   ...Text...
3 </div>
4 <style>
5   #example {
6     width: 300px;
7     height: 200px;
8     border: 25px solid #E8C48F;
9     padding: 20px;
10    overflow: auto;
11  }
12 </style>
```



9. DOM: Element size & scrolling

Geometry



9. DOM: Element size & scrolling

Summary

Elements have the following geometry properties:

- `offsetParent` – is the nearest positioned ancestor or `td`, `th`, `table`, `body`.
- `offsetLeft/offsetTop` – coordinates relative to the upper-left edge of `offsetParent`.
- `offsetWidth/offsetHeight` – “outer” width/height of an element including borders.
- `clientLeft/clientTop` – the distances from the upper-left outer corner to the upper-left inner (content + padding) corner. For left-to-right OS they are always the widths of left/top borders. For right-to-left OS the vertical scrollbar is on the left so `clientLeft` includes its width too.
- `clientWidth/clientHeight` – the width/height of the content including paddings, but without the scrollbar.
- `scrollWidth/scrollHeight` – the width/height of the content, just like `clientWidth/clientHeight`, but also include scrolled-out, invisible part of the element.
- `scrollLeft/scrollTop` – width/height of the scrolled out upper part of the element, starting from its upper-left corner.

All properties are read-only except `scrollLeft/scrollTop` that make the browser scroll the element if changed.

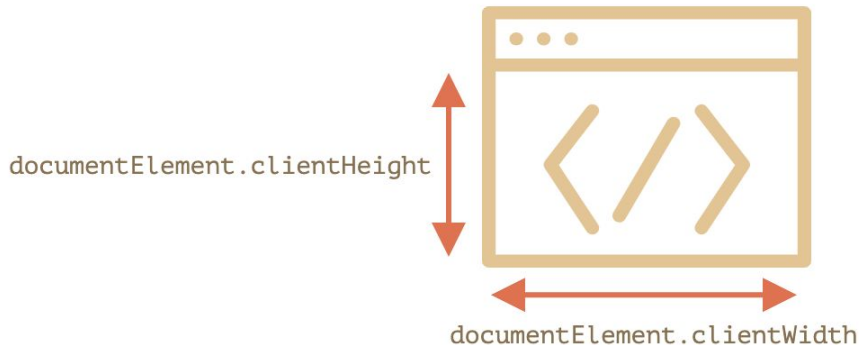
10. DOM: Window sizes & scrolling

How do we find the width and height of the browser window? How do we get the full width and height of the document, including the scrolled out part? How do we scroll the page using JavaScript?

For this type of information, we can use the root document element `document.documentElement`, that corresponds to the `<html>` tag. But there are additional methods and peculiarities to consider.

Width/height of the window

To get window width and height, we can use the `clientWidth/clientHeight` of `document.documentElement`:



10. DOM: Window sizes & scrolling

Summary

Geometry:

- Width/height of the visible part of the document (content area width/height):
`document.documentElement.clientWidth/clientHeight`
- Width/height of the whole document, with the scrolled out part:

```
let scrollHeight = Math.max(  
  document.body.scrollHeight, document.documentElement.scrollHeight,  
  document.body.offsetHeight, document.documentElement.offsetHeight,  
  document.body.clientHeight, document.documentElement.clientHeight  
);  
  
alert('Full document height, with scrolled out part: ' + scrollHeight);
```


10. DOM: Window sizes & scrolling

Summary

Scrolling:

- Read the current scroll: `window.pageYOffset/pageXOffset`.
- Change the current scroll:
 - `window.scrollTo(pageX,pageY)` – absolute coordinates,
 - `window.scrollBy(x,y)` – scroll relative the current place,
 - `elem.scrollToView(top)` – scroll to make `elem` visible (align with the top/bottom of the window).

11. DOM: Coordinates

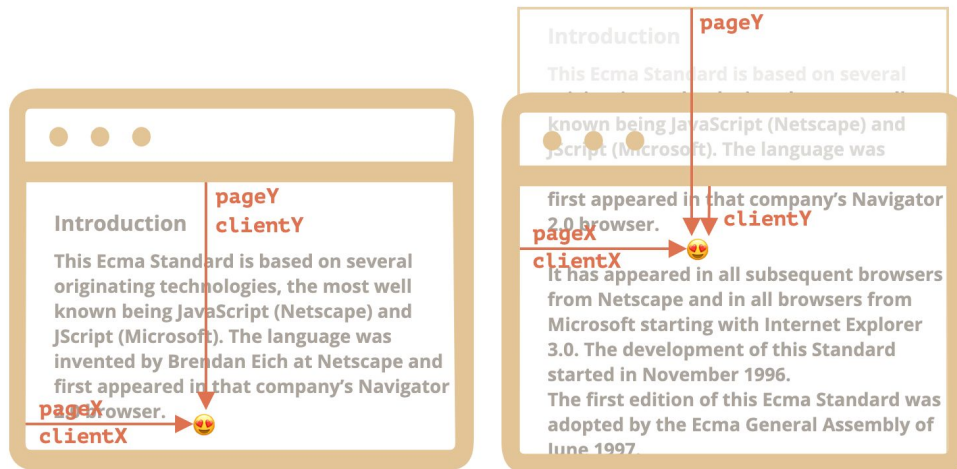
To move elements around we should be familiar with coordinates.

Most JavaScript methods deal with one of two coordinate systems:

1. **Relative to the window** – similar to `position:fixed`, calculated from the window top/left edge.
 - we'll denote these coordinates as `clientX/clientY`, the reasoning for such name will become clear later, when we study event properties.
2. **Relative to the document** – similar to `position:absolute` in the document root, calculated from the document top/left edge.
 - we'll denote them `pageX/pageY`.

11. DOM: Coordinates

On this picture we take a point in the document and demonstrate its coordinates before the scroll (left) and after it (right):



When the document scrolled:

- **pageY** – document-relative coordinate stayed the same, it's counted from the document top (now scrolled out).
- **clientY** – window-relative coordinate did change (the arrow became shorter), as the same point became

11. DOM: Coordinates

Element coordinates: `getBoundingClientRect`

The method `elem.getBoundingClientRect()` returns window coordinates for a minimal rectangle that encloses `elem` as an object of built-in `DOMRect` class.

Main `DOMRect` properties:

- `x/y` – X/Y-coordinates of the rectangle origin relative to window,
- `width/height` – width/height of the rectangle (can be negative).

Additionally, there are derived properties:

- `top/bottom` – Y-coordinate for the top/bottom rectangle edge,
- `left/right` – X-coordinate for the left/right rectangle edge.

11. DOM: Coordinates

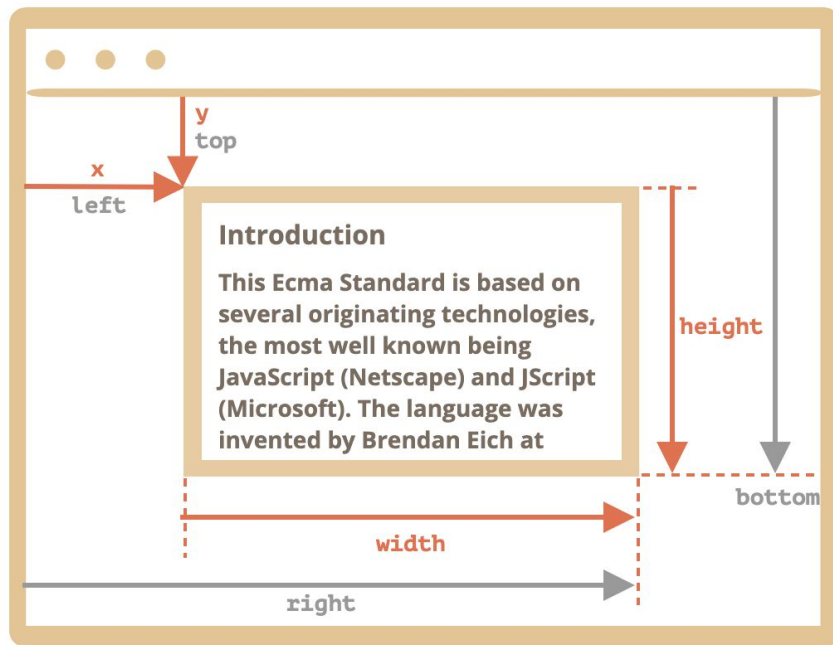
Element coordinates: `getBoundingClientRect`

As you can see,

`x/y` and `width/height` fully describe the rectangle.

Derived properties can be easily calculated from them:

- `left = x`
- `top = y`
- `right = x + width`
- `bottom = y + height`



11. DOM: Coordinates

elementFromPoint(x, y)

The call to `document.elementFromPoint(x, y)` returns the most nested element at window coordinates `(x, y)`.

The syntax is:

```
let elem = document.elementFromPoint(x, y);
```

For instance, the code below highlights and outputs the tag of the element that is now in the middle of the window:

```
1 let centerX = document.documentElement.clientWidth / 2;  
2 let centerY = document.documentElement.clientHeight / 2;  
3  
4 let elem = document.elementFromPoint(centerX, centerY);  
5  
6 elem.style.background = "red";  
7 alert(elem.tagName);
```



11. DOM: Coordinates

Document coordinates

Document-relative coordinates start from the upper-left corner of the document, not the window.

In CSS, window coordinates correspond to `position:fixed`, while document coordinates are similar to `position:absolute` on top.

We can use `position:absolute` and `top/left` to put something at a certain place of the document, so that it remains there during a page scroll. But we need the right coordinates first.

There's no standard method to get the document coordinates of an element. But it's easy to write it.

The two coordinate systems are connected by the formula:

- $\text{pageY} = \text{clientY} + \text{height of the scrolled-out vertical part of the document.}$
- $\text{pageX} = \text{clientX} + \text{width of the scrolled-out horizontal part of the document.}$

11. DOM: Coordinates

Document coordinates

The function `getCoords(elem)` will take window coordinates from `elem.getBoundingClientRect()` and add the current scroll to them:

```
1 // get document coordinates of the element
2 function getCoords(elem) {
3   let box = elem.getBoundingClientRect();
4
5   return {
6     top: box.top + window.pageYOffset,
7     right: box.right + window.pageXOffset,
8     bottom: box.bottom + window.pageYOffset,
9     left: box.left + window.pageXOffset
10  };
11 }
```


11. DOM: Coordinates

Summary

Any point on the page has coordinates:

1. Relative to the window – `elem.getBoundingClientRect()`.
2. Relative to the document – `elem.getBoundingClientRect()` plus the current page scroll.

Window coordinates are great to use with `position:fixed`, and document coordinates do well with `position:absolute`.

Both coordinate systems have their pros and cons; there are times we need one or the other one, just like CSS `position absolute` and `fixed`.

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Lab

Number Guesser & Exploration Game

Homework

Browser App using DOM & Events